

Modül 9

Forms

Template-driven, Reactive, ve Signal Forms

Modül:	9 / 30
Ders Sayısı:	9 ders
Seviye:	Orta-İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 9'a Hoş Geldin

Form'lar — kullanıcı etkileşiminin kalbi. Angular'da 4 farklı yaklaşım var: manuel binding (Modül 4'te yaptığımız), template-driven (ngModel), reactive forms (FormControl), ve gelecek olan Signal Forms.

Bu modülün sonunda yapabileceklerin:

- ✓ 4 form yaklaşımının farkını net bilmek
- ✓ Reactive Forms ile complex form'lar yazabilmek
- ✓ Built-in ve custom validator'lar kullanmak
- ✓ Cross-field validation ve async validation
- ✓ Dynamic forms ve FormArray
- ✓ Custom Form Control (ControlValueAccessor)
- ✓ Signal Forms'un gelecekteki yapısını anlamak

4 Form Yaklaşımı

Hangi durumda hangisini kullanacağını bilmek için önce hepsini tanımalıyız.

1 Manuel Binding (Modül 4 Pattern)

```
@Component({...})
export class FormComponent {
  title = signal('');
  isValid = computed(() => this.title().trim().length > 0);

  onInput(e: Event) {
    this.title.set((e.target as HTMLInputElement).value);
  }
}

<input [value]="title()" (input)="onInput($event)">
```

- ✓ ✓ Basit, küçük form'lar için ideal
- ✓ ✓ Tam signal-based, modern
- ✓ ✗ Karmaşık validation için yetersiz

2 Template-Driven (ngModel)

```
import { FormsModule } from '@angular/forms';

@Component({
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="title" #titleInput="ngModel" required>
    @if (titleInput.invalid && titleInput.touched) {
      <span>Zorunlu</span>
    }
  `
})
export class FormComponent {
  title = '';
}
```

- ✓ ✓ Çok hızlı yazılır
- ✓ ✓ Basit form'lar için OK
- ✓ ✗ Signal uyumsuz (eski yaklaşım)
- ✓ ✗ Test etmesi zor

3 Reactive Forms (FormControl)

```
import { ReactiveFormsModule, FormControl, FormGroup, Validators } from
 '@angular/forms';

@Component({
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="onSubmit()">
      <input formControlName="title">
      <input formControlName="email">
      <button [disabled]="form.invalid">Submit</button>
    </form>
  `,
})
export class FormComponent {
  form = new FormGroup({
    title: new FormControl('', [Validators.required, Validators.minLength(3)]),
    email: new FormControl('', [Validators.email]),
  });

  onSubmit() {
    if (this.form.valid) {
      console.log(this.form.value);
    }
  }
}
```

- ✓ ✓ Karmaşık form'lar için ideal
- ✓ ✓ Programmatic, test edilebilir
- ✓ ✓ Validation, cross-field, async destekli
- ✓ ✗ Signal ile uyum biraz extra çaba gerektirir

4▯ Signal Forms (Gelecek)

Angular ekibi Signal Forms üzerinde çalışıyor. Henüz stable değil ama gelecekte standard olacak. Şimdilik Reactive Forms + toSignal() ile köprü kuruyoruz.

▯ Hangisi Ne Zaman?

Senaryo	Yaklaşım	Sebep
Search input	Manuel binding	Tek field, hızlı
Login form	Reactive	Validation, test
Multi-step form	Reactive	Karmaşık state
Dynamic fields	Reactive (FormArray)	Programmatic
Simple settings	Manuel binding	Signal'lı, sade

Template-Driven Forms

ngModel ile çalışmak. Hızlı ama eski yaklaşım.

□ Temel Kullanım

```
import { FormsModule } from '@angular/forms';

@Component({
  imports: [FormsModule],
  template: `
    <form #f="ngForm" (ngSubmit)="onSubmit(f)">
      <input
        name="username"
        [(ngModel)]="user.username"
        required
        minlength="3"
        #username="ngModel">

      @if (username.invalid && username.touched) {
        <div>
          @if (username.errors?.['required']) {
            <span>Username zorunlu</span>
          }
          @if (username.errors?.['minlength']) {
            <span>En az 3 karakter</span>
          }
        </div>
      }

      <button [disabled]="f.invalid">Submit</button>
    </form>
  `
})
export class FormComponent {
  user = { username: '', email: '' };

  onSubmit(form: NgForm) {
    if (form.valid) {
      console.log(this.user);
    }
  }
}
```

⚠ Önemli Not

⚠ **Signal Uyumsuzluđu**

ngModel signal'ları desteklemiyor. Eđer modern signal-based mimari kullanıyorsan, Reactive Forms veya manuel binding tercih et. Template-driven sadece legacy kod için.

Reactive Forms

FormControl, FormGroup, FormBuilder ile profesyonel form'lar.

□ FormGroup ve FormControl

```
import { FormControl, FormGroup, Validators, ReactiveFormsModule } from
 '@angular/forms';

@Component({
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="onSubmit()">
      <input formControlName="name">
      <input formControlName="email">
      <input formControlName="age" type="number">
      <button>Submit</button>
    </form>
  `,
})
export class UserFormComponent {
  form = new FormGroup({
    name: new FormControl('', { nullable: true, validators: [Validators.required]
  }),
    email: new FormControl('', [Validators.required, Validators.email]),
    age: new FormControl<number | null>(null, [Validators.min(18)]),
  });

  onSubmit() {
    if (this.form.valid) {
      console.log(this.form.value);
      console.log(this.form.getRawValue()); // disabled fields dahil
    }
  }
}
```

□ FormBuilder — Shorthand

```

import { FormBuilder, Validators } from '@angular/forms';

@Component({...})
export class UserFormComponent {
  private fb = inject(FormBuilder);

  // Daha kısa
  form = this.fb.group({
    name: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]],
    age: [null as number | null, [Validators.min(18)]],
  });

  // nonNullable group
  form2 = this.fb.nonNullable.group({
    name: ['', Validators.required],
    email: '',
  });
}

```

❑ FormControl API

```

const control = new FormControl('initial');

// Değer
control.value;           // 'initial'
control.setValue('x');   // Direkt
control.patchValue('x'); // Aynı

// State
control.valid;           // boolean
control.invalid;         // boolean
control.dirty;           // Kullanıcı değiştirdi mi
control.pristine;        // Hiç değişmedi mi
control.touched;         // Focus aldı mı
control.untouched;       // Hiç focus almadı mı

// Errors
control.errors;          // { required: true, minlength: {...} }
control.hasError('required');

// Disable
control.disable();
control.enable();

// Reset
control.reset();
control.reset('default-value');

// Observable
control.valueChanges.subscribe(value => { ... });
control.statusChanges.subscribe(status => { ... });

```


Validators

Built-in, custom, async, ve cross-field validation.

□ Built-in Validators

```
import { Validators } from '@angular/forms';

new FormControl('', [
  Validators.required,
  Validators.email,
  Validators.minLength(3),
  Validators.maxLength(50),
  Validators.min(18),
  Validators.max(120),
  Validators.pattern(/^[a-zA-Z]+$/),
  Validators.requiredTrue, // checkbox için
]);
```

□ Custom Sync Validator

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

// Custom validator factory
export function forbiddenName(forbidden: string): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    return control.value === forbidden
      ? { forbidden: { value: control.value } }
      : null;
  };
}

// Kullanım
new FormControl('', [Validators.required, forbiddenName('admin')]);
```

□ Custom Async Validator

```

import { AsyncValidatorFn } from '@angular/forms';
import { map, debounceTime, switchMap, of } from 'rxjs';

export function uniqueUsername(api: UserApiService): AsyncValidatorFn {
  return (control: AbstractControl) => {
    return of(control.value).pipe(
      debounceTime(500),
      switchMap(value => api.checkUsername(value)),
      map(isUnique => isUnique ? null : { notUnique: true })
    );
  };
}

// Kullanım
new FormControl('', {
  validators: [Validators.required],
  asyncValidators: [uniqueUsername(inject(UserApiService))]
});

```

❑ Cross-Field Validation

```

// FormGroup level validator
export const passwordMatch: ValidatorFn = (group: AbstractControl): ValidationErrors
| null => {
  const password = group.get('password')?.value;
  const confirm = group.get('confirmPassword')?.value;
  return password === confirm ? null : { passwordMismatch: true };
};

// Kullanım
this.form = this.fb.group({
  password: ['', Validators.required],
  confirmPassword: ['', Validators.required],
}, { validators: passwordMatch });

// Template
@if (form.errors?.['passwordMismatch']) {
  <span>Şifreler eşleşmiyor</span>
}

```

Form State & Error Display

Validation'ı kullanıcıya doğru zamanda göstermek.

□ State Yönetimi

```
<form [formGroup]="form">
  <input formControlName="email">

  <!-- Sadece touched ise göster (kullanıcı focus aldı ve çıktı) -->
  @let emailControl = form.get('email');
  @if (emailControl?.invalid && emailControl?.touched) {
    @if (emailControl?.errors?.['required']) {
      <span>Email zorunlu</span>
    }
    @if (emailControl?.errors?.['email']) {
      <span>Geçerli email gir</span>
    }
  }
</form>
```

□ Reusable FormErrors Component

```

@Component({
  selector: 'app-form-errors',
  template: `
    @if (control().invalid && control().touched) {
      <div class="errors">
        @for (error of errors(); track error.key) {
          <span class="error">{{ error.message }}</span>
        }
      </div>
    }
  `,
})
export class FormErrorsComponent {
  control = input.required<AbstractControl>();

  errors = computed(() => {
    const errs = this.control().errors;
    if (!errs) return [];

    const messages: Record<string, string> = {
      required: 'Bu alan zorunlu',
      email: 'Geçerli email gir',
      minlength: `En az ${errs['minlength']?.requiredLength} karakter`,
      passwordMismatch: 'Şifreler eşleşmiyor',
    };

    return Object.keys(errs).map(key => ({
      key,
      message: messages[key] || 'Hatalı'
    }));
  });
}

// Kullanım
<input formControlName="email">
<app-form-errors [control]="form.get('email')!" />

```

Dynamic Forms — FormArray

Dinamik olarak field ekleyip kaldırmak için.

FormArray Kullanımı

```
import { FormArray, FormBuilder } from '@angular/forms';

@Component({...})
export class TaskListFormComponent {
  private fb = inject(FormBuilder);

  form = this.fb.group({
    title: [''],
    tasks: this.fb.array([
      this.fb.control('', Validators.required)
    ])
  });

  get tasks() {
    return this.form.get('tasks') as FormArray;
  }

  addTask() {
    this.tasks.push(this.fb.control('', Validators.required));
  }

  removeTask(index: number) {
    this.tasks.removeAt(index);
  }
}
```

```
<form [formGroup]="form">
  <input formControlName="title">

  <div formArrayName="tasks">
    @for (task of tasks.controls; track $index; let i = $index) {
      <div>
        <input [formControlName]="i">
        <button type="button" (click)="removeTask(i)">Sil</button>
      </div>
    }
  </div>

  <button type="button" (click)="addTask()">Task Ekle</button>
</form>
```

Reactive Forms + Signals

Reactive Form'ları signal'larla bağlama.

toSignal Köprüsü

```
@Component({...})
export class FormComponent {
  private fb = inject(FormBuilder);

  form = this.fb.group({
    search: [''],
    sortBy: ['name'],
  });

  // FormControl değerlerini signal olarak izle
  search = toSignal(this.form.controls.search.valueChanges, {
    initialValue: ''
  });
  sortBy = toSignal(this.form.controls.sortBy.valueChanges, {
    initialValue: 'name'
  });

  // Tüm form değeri
  formValue = toSignal(this.form.valueChanges, {
    initialValue: this.form.value
  });

  // Status (valid/invalid)
  formStatus = toSignal(this.form.statusChanges, {
    initialValue: this.form.status
  });

  // Bunlar şimdi diğer computed/effect'lerde kullanılabilir
  results = httpResource<Result[]>(() =>
    `/api/search?q=${this.search()}&sort=${this.sortBy()}`
  );
}
```

Custom Form Control — ControlValueAccessor

Kendi component'ini reactive form'a entegre etmek.

❑ Hangi Problem?

Custom bir component yazdın (örn: app-color-picker). Bunu formControlName="color" ile reactive form'a bağlamak istiyorsun.

❑ Component Tanımı

```
import { Component, forwardRef, signal } from '@angular/core';
import { ControlValueAccessor, NG_VALUE_ACCESSOR } from '@angular/forms';

@Component({
  selector: 'app-color-picker',
  template: `
    <div class="picker">
      @for (color of colors; track color) {
        <button
          [style.background]="color"
          [class.selected]="value() === color"
          (click)="selectColor(color)">
        </button>
      }
    </div>
  `,
  providers: [
    {
      provide: NG_VALUE_ACCESSOR,
      useExisting: forwardRef(() => ColorPickerComponent),
      multi: true
    }
  ]
})
export class ColorPickerComponent implements ControlValueAccessor {
  colors = ['red', 'green', 'blue', 'yellow'];
  value = signal<string>('');
  disabled = signal(false);
  // ... (devamı sonraki bloğta)
}
```

❑ ControlValueAccessor Methodları

```

private onChange: (value: string) => void = () => {};
private onTouched: () => void = () => {};

// Form'dan değer geldiğinde
writeValue(value: string) {
    this.value.set(value);
}

// Değer değiştiğinde form'a bildirmek için
registerOnChange(fn: (value: string) => void) {
    this.onChange = fn;
}

registerOnTouched(fn: () => void) {
    this.onTouched = fn;
}

setDisabledState(disabled: boolean) {
    this.disabled.set(disabled);
}

selectColor(color: string) {
    if (this.disabled()) return;
    this.value.set(color);
    this.onChange(color); // Form'a bildir
    this.onTouched();
}

```

Kullanım:

```

form = this.fb.group({
    favoriteColor: ['red']
});

// Template
<form [formGroup]="form">
    <app-color-picker formControlName="favoriteColor"></app-color-picker>
</form>

```


Signal Forms — Gelecek

Angular ekibi yeni bir form API'sı geliştiriyor. İlk bakış.

□ Vizyon

Signal Forms — Reactive Forms'un signal-native versiyonu. Henüz preview'da ama gelecekteki standard olacak. Avantajlar:

- ✓ Tam signal-based
- ✓ Type-safe by default
- ✓ Daha az boilerplate
- ✓ Reactive Forms ile koexistence

□ Şimdilik Strateji

□ Pratik Yaklaşım

Bugün: Reactive Forms + toSignal() köprüsü

Yakın gelecek: Signal Forms stable olunca migration

Geçiş kolay: Reactive Forms'ı iyi öğrenmek temel olacak

Modül 9 Özeti

Form'lar konusunda artık tam donanımlısın. Hangi yaklaşımı ne zaman kullanacağını biliyorsun. Validation, dynamic forms, custom controls — hepsi alet kutunda.

Neler Öğrendin?

- ✓ 4 form yaklaşımı arasında doğru seçim
- ✓ Reactive Forms: FormControl, FormGroup, FormBuilder
- ✓ Built-in ve custom validators (sync + async)
- ✓ Cross-field validation
- ✓ Error display patterns
- ✓ Dynamic forms: FormArray
- ✓ Reactive Forms + Signals köprüsü
- ✓ Custom Form Control: ControlValueAccessor

Sıradaki Modül

Modül 10 — HttpClient & API Communication. HTTP istekleri, interceptor'lar, error handling, file upload/download, production patterns.